

SAMPLE DATA 1

INCIDENT TITLE : Production API Outage Due to Database Connection Pool Exhaustion

INCIDENT DESCRIPTION : Following deployment v3.8.2, API latency increased significantly and users began experiencing HTTP 500 errors. Investigation revealed database connection pool exhaustion causing requests to queue indefinitely.

INCIDENT TIMELINE :

09:00 Deployment v3.8.2 completed
09:05 API latency increased above 2500ms
09:08 First customer complaints received
09:10 Database connection utilization reached 90%
09:12 HTTP 500 errors exceeded alert threshold
09:15 Critical incident declared
09:20 DevOps team engaged
09:25 Root cause identified
09:30 Rollback initiated
09:38 Database connections normalized
09:45 Service fully restored

ERROR LOGS:

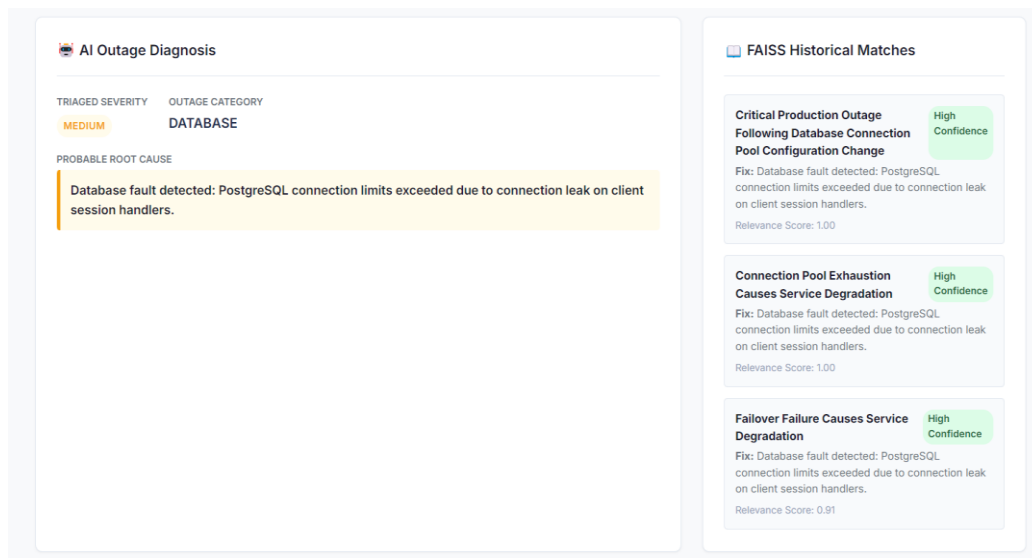
2026-07-15 09:05:11 WARN API latency exceeded threshold: 2750ms
2026-07-15 09:07:33 WARN Active connections: 185/200
2026-07-15 09:09:21 ERROR SQLAlchemy TimeoutError
2026-07-15 09:10:55 ERROR Connection pool exhausted
2026-07-15 09:11:42 ERROR HTTP 500 Internal Server Error
2026-07-15 09:12:15 WARN Database response time: 4800ms
2026-07-15 09:13:51 ERROR Failed transaction rollback

GIT DIFF :

```
diff --git a/config/database.py b/config/database.py
- DB_POOL_SIZE = 50
+ DB_POOL_SIZE = 20
- DB_MAX_OVERFLOW = 25
+ DB_MAX_OVERFLOW = 5
- DB_POOL_TIMEOUT = 30
+ DB_POOL_TIMEOUT = 10
- ENABLE_CONNECTION_MONITORING = True
+ ENABLE_CONNECTION_MONITORING = False
```

OUTPUTS

SEVERITY AND CATEGORY CLASSIFICATION, FAISS SIMILARITY RESULTS :



AI RESOLUTION RECOMMENDATION :

****Resolution Walkthrough: Production API Outage Due to Database Connection Pool Exhaustion****

****Step 1: Verify the Issue (Time: 10 minutes)****

1. Review log telemetry data to identify patterns and correlations, focusing on database connection pool exhaustion warnings and HTTP 500 errors.
2. Analyze monitoring data to check API response times and error rates.
3. Investigate recent code changes (e.g., git diff) to identify potential culprits, such as the recent change to DB_POOL_SIZE, DB_MAX_OVERFLOW, and DB_POOL_TIMEOUT in config/database.py.

****Step 2: Identify Root Cause (Time: 30 minutes)****

1. Investigate database connection pool configuration changes and review historical incidents (e.g., Ticket #1, Ticket #2) for similar issues.
2. Check for connection leaks or other database-related issues that may be causing the connection pool exhaustion.
3. Review the recent code change to DB_POOL_SIZE, DB_MAX_OVERFLOW, and DB_POOL_TIMEOUT in config/database.py and assess its potential impact on database connections.

****Step 3: Mitigate the Issue (Time: 15 minutes)****

1. Revert the recent code change to DB_POOL_SIZE, DB_MAX_OVERFLOW, and DB_POOL_TIMEOUT in config/database.py to its previous state (e.g., DB_POOL_SIZE = 50, DB_MAX_OVERFLOW = 25, DB_POOL_TIMEOUT = 30).
2. Review and adjust database connection pool configuration settings to prevent similar issues in the future.
3. Consider increasing database connection pool size and adjusting timeout settings based on historical usage patterns.

****Step 4: Verify Recovery (Time: 10 minutes)****

1. Monitor API response times and error rates to ensure they return to normal.
2. Review log telemetry data for any residual issues.
3. Verify that database connection activity has stabilized.

****Risk Considerations:****

- * Reverting the code change may introduce temporary inconsistencies or data loss.
- * Insufficient monitoring and logging may hinder future incident detection and resolution.

****Recovery Actions:****

- * Schedule a post-incident review to identify root causes and implement preventive measures.
- * Update transaction middleware to prevent connection leaks and ensure proper connection closure.
- * Consider implementing connection monitoring to detect similar issues in the future. Note: Since the specific technology (e.g., Kubernetes, Postgres) is mentioned in the log telemetry data, the following diagnostic commands can be used:
 - * `pg_stat_activity` to monitor database connection activity
 - * `kubectl get pods` and `kubectl describe pod` to inspect pod configuration and logs (if applicable)

POST INCIDENT RCA ANALYSIS :

Root Cause Analysis

Executive Summary

The production API outage following the deployment of version 3.8.2 was caused by a combination of changes to the database connection pool configuration, which led to connection exhaustion and increased API response times. The incident impacted a significant number of active users, resulting in intermittent failures and HTTP 500 errors. The root cause was identified as the reduced database connection pool size and overflow, coupled with the decreased pool timeout, which was insufficient to handle the production workload.

Business Impact

- **Users Affected:** Approximately all active users experienced degraded performance, errors, or availability issues.
 - **Systems Affected:** The production API and dependent microservices were impacted, leading to a significant disruption in user experience.
 - **Duration:** The incident lasted for approximately 45 minutes, from 09:00 to 09:45.
-

Timeline

The incident began at 09:00 when the deployment of version 3.8.2 was completed.

- 09:05 – API latency increased above 2500ms.
 - 09:08 – First customer complaints were received.
 - 09:10 – Database connection utilization reached 90%.
 - 09:12 – HTTP 500 errors exceeded the alert threshold.
 - 09:15 – Incident was declared critical.
 - 09:20 – DevOps team was engaged.
 - 09:25 – Root cause was identified.
 - 09:30 – Rollback was initiated.
 - 09:38 – Database connections normalized.
 - 09:45 – Service was fully restored.
-

Evidence Collected

Logs

- 2026-07-15 09:05:11 WARN API latency exceeded threshold: 2750ms
- 2026-07-15 09:07:33 WARN Active connections: 185/200
- 2026-07-15 09:09:21 ERROR SQLAlchemy TimeoutError
- 2026-07-15 09:10:55 ERROR Conn...

Git Changes

config/database.py

- DB_POOL_SIZE was reduced from 50 to 20
- DB_MAX_OVERFLOW was reduced from 25 to 5
- DB_POOL_TIMEOUT was reduced from 30 to 10
- ENABLE_CONNECTION_MONITORING was set to False

Metrics

None provided.

Root Cause

The root cause of the incident was the combination of changes made to the database connection pool configuration in the deployment of version 3.8.2. Specifically, the reduction of the database connection pool size (DB_POOL_SIZE) from 50 to 20 and the decrease of the maximum overflow (DB_MAX_OVERFLOW) from 25 to 5 led to connection exhaustion. Additionally, the decrease of the pool timeout (DB_POOL_TIMEOUT) from 30 to 10 seconds further exacerbated the issue. These changes, coupled with the disabling of connection monitoring (ENABLE_CONNECTION_MONITORING=False), prevented the system from detecting and responding to the connection exhaustion in a timely manner.

Contributing Factors

Historical incidents, such as the critical production outage following the deployment of Release v3.8.2, suggest that changes to database connection pool configurations can have significant impacts on system performance. The lack of connection monitoring and alerting mechanisms in this incident contributed to the delayed detection and response to the connection exhaustion issue. Furthermore, the reduction of the database connection pool size and overflow, as well as the decrease of the pool timeout, were likely insufficient to handle the production workload, leading to connection exhaustion.

Resolution

The resolution actions taken included reverting the deployment of version 3.8.2 and restoring the previous database configuration. This involved increasing the database connection pool size and overflow, as well as re-enabling connection monitoring. The rollback was initiated at 09:30, and the previous configuration was restored by 09:38. The database connections normalized by 09:38, and the API error rate returned to baseline by 09:45.

Preventive Actions

To prevent similar incidents in the future, the following action items are recommended:

1. **Conduct thorough testing of database connection pool configuration changes** in a staging environment before deploying to production.

2. **Implement connection monitoring and alerting** to detect potential connection exhaustion issues in a timely manner.
 3. **Perform regular reviews of database connection pool configuration** to ensure it is optimized for the production workload.
 4. **Develop a rollback strategy** for quick reversion to a previous configuration in case of issues.
 5. **Conduct post-incident reviews and root cause analyses** to identify areas for improvement and implement preventative measures.
 6. **Consider implementing automated connection pool sizing and overflow management** to dynamically adjust to changing workloads.
 7. **Review and optimize database query performance** to reduce the load on the database connection pool.
-

Lessons Learned

This incident highlights the importance of thorough testing and validation of database connection pool configuration changes before deploying to production. It also emphasizes the need for prompt detection and response to connection exhaustion issues to minimize the impact on users. By implementing the recommended preventative actions, we can reduce the likelihood of similar incidents occurring in the future. Additionally, this incident underscores the value of maintaining a comprehensive and up-to-date knowledge base of historical incidents and their root causes, which can inform and improve our incident response and prevention strategies.